

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по домашней работе № 5

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

Межсервисное взаимодействие через RabbitMQ

Вариант 5

Выполнил: студент группы БР1.1

Кошкарев Кирилл Павлович

Проверил:

Санкт-Петербург, 2026

1. Цель работы

Подключить и настроить брокер сообщений RabbitMQ и реализовать межсервисное взаимодействие через очереди в соответствии с контрактом событий из ДЗ4.

2. Топология обмена

Используется один durable topic-exchange **recipes.events**. Каждый потребитель объявляет собственную durable-очередь и привязывает её по шаблонам ключей маршрутизации, поэтому добавление нового потребителя не затрагивает существующие.

Таблица 1 — Очереди и их привязки

Очередь	Потребитель	Привязки	Назначение
social.projections	social-service	user.*, recipe.*	поддержание проекций user_projection и recipe_projection, каскадная очистка
recipe.counters	recipe-service	social.recipe.*, social.comment.*	обновление денормализованных счётчиков likes_count и comments_count
*.dead	—	через recipes.events.dlx	сообщения, которые не удалось обработать

3. Конверт сообщения

Все события передаются в едином конверте: это позволяет обрабатывать идемпотентность и версионирование единообразно, не разбираясь в структуре конкретного события.

```
export interface EventEnvelope<T = Record<string, unknown>> {  
  eventId: string;      // uuid — ключ идемпотентности  
  type: string;         // routing key, например recipe.published  
  occurredAt: string;   // ISO-8601, используется для защиты от устаревших событий  
  producer: string;     // имя сервиса-издателя  
  version: number;      // версия схемы полезной нагрузки  
  payload: T;  
}
```

Листинг 1 — конверт события

4. Публикация событий

Событие публикуется после успешной фиксации изменений в базе, поэтому подписчики никогда не получают уведомление о том, чего не произошло. Сообщения помечаются как persistent и переживают перезапуск

брокера.

```
async publish(type: string, payload: Record<string, unknown>): Promise<boolean> {
  const envelope: EventEnvelope = {
    eventId: randomUUID(), type, occurredAt: new Date().toISOString(),
    producer: this.options.producer, version: 1, payload,
  };
  if (!this.channel) {
    this.log(`событие ${type} не отправлено: нет соединения с брокером`);
    return false; // сценарий пользователя не прерывается
  }
  return this.channel.publish(
    this.options.exchange, type, Buffer.from(JSON.stringify(envelope)),
    { persistent: true, contentType: 'application/json', messageId: envelope.eventId },
  );
}
```

Листинг 2 — публикация события в шине

5. Обработка событий и идемпотентность

Брокер гарантирует доставку не менее одного раза, поэтому потребитель обязан быть устойчив к повторам. Реализованы две меры: журнал обработанных событий `processed_events` и изменение счётчиков относительной дельтой в одном SQL-выражении.

```
const alreadyProcessed = async (event: EventEnvelope): Promise<boolean> => {
  const repo = AppDataSource.getRepository(ProcessedEvent);
  if (await repo.findOne({ where: { eventId: event.eventId } })) return true;
  await repo.save(repo.create({ eventId: event.eventId, type: event.type }));
  return false;
};

const applyDelta = async (recipeId: number, column: string, delta: number) => {
  // Одним атомарным UPDATE: не перетираем параллельные изменения и не уходим ниже нуля.
  await AppDataSource.getRepository(Recipe).query(
    `UPDATE recipes SET "${column}" = GREATEST("${column}" + $2, 0) WHERE id = $1`,
    [recipeId, delta],
  );
};
```

Листинг 3 — защита от повторной доставки и атомарное изменение счётчика

Проекции дополнительно защищены от нарушения порядка: перед записью сравнивается отметка `occurredAt` пришедшего события с сохранённой `synced_at`, и более старое событие отбрасывается.

6. Отказоустойчивость

- **Недоступность брокера не останавливает сервис.** При старте выполняется попытка подключения; при неудаче сервис продолжает обслуживать HTTP-запросы, а подключение восстанавливается в фоне с экспоненциальной паузой (до 30 секунд);

- **Обрыв соединения** обрабатывается событием `close`: подписки восстанавливаются автоматически после переподключения;
- **Ошибка обработки** подтверждается методом `nack` с `requeue = false`, и сообщение уходит в `dead-letter`-очередь, а не закидывается на одном потребителе;
- **Отставание проекций** компенсируется синхронным вызовом `/internal/recipes/{id}/summary`.

7. Настройка брокера

Брокер поднимается в контуре `docker compose` (JP3) образом `rabbitmq:4-management-alpine` с постоянным томом и проверкой состояния. Веб-консоль доступна на порту 15672.

```
rabbitmq:
  image: rabbitmq:4-management-alpine
  environment:
    RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER:-lecters}
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD:-lecters}
  volumes:
    - rabbitmq-data:/var/lib/rabbitmq
  ports: ["5672:5672", "15672:15672"]
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
    interval: 10s
    retries: 10
    start_period: 20s
```

Листинг 4 — описание брокера в `docker compose`

8. Проверка топологии

После запуска контура и наполнения демонстрационными данными состояние брокера соответствует проекту: объявлены обменник и его `dead-letter`-пара, две рабочие очереди с одним потребителем каждая и корректные привязки.

```

$ docker compose exec rabbitmq rabbitmqctl list_exchanges name type
name                                type
recipes.events                     topic
recipes.events.dlx                 topic

$ docker compose exec rabbitmq rabbitmqctl list_queues name messages consumers
name                                messages consumers
recipe.counters                     0                1
recipe.counters.dead                 0                0
social.projections                  0                1
social.projections.dead              0                0

$ docker compose exec rabbitmq rabbitmqctl list_bindings source_name routing_key destination_name
recipes.events                     recipe.*          social.projections
recipes.events                     user.*           social.projections
recipes.events                     social.recipe.*   recipe.counters
recipes.events                     social.comment.*  recipe.counters
recipes.events.dlx                 social.projections.# social.projections.dead
recipes.events.dlx                 recipe.counters.# recipe.counters.dead

```

Листинг 5 — состояние брокера после запуска контура

9. Проверка обмена событиями

Наполнение демонстрационными данными сразу показывает работу шины: `user-service` и `recipe-service` публикуют события, а `social-service` наполняет по ним проекции.

```

$ docker compose exec user-service node dist/seed.js
[bus:user-service] подключено к amqp://lecters:lecters@rabbitmq:5672, exchange «recipes.events»
[bus:user-service] → user.registered {"userId":1,"username":"dr_lecter",...}
[seed] user-service: 4 пользователя, 3 подписки

$ docker compose exec recipe-service node dist/seed.js
[bus:recipe-service] → recipe.published {"recipeId":6,"authorId":2,"title":"Запечённый фенхель..."}
[seed] recipe-service готов

$ curl -s localhost:3000/health | jq '.services["social-service"].projections'
{ "recipes": 6, "users": 4 }

```

Листинг 6 — наполнение проекций событиями

Для регрессионной проверки написан сценарий `events-smoke.mjs` из 22 проверок. Он проверяет весь цикл: от появления проекции до каскадного удаления данных. Существенно, что счётчики рецепта проверяются *после* действий в `social-service`: изменить их можно только событием, прямого доступа к чужой базе у сервиса нет.

Отдельного внимания потребовал сам способ наблюдения за проекциями. Первая версия сценария сравнивала число строк в проекции до и после регистрации пользователя, и проверка начала падать, хотя событие обрабатывалось корректно. Причина в том, что базы сервисов наполняются демонстрационными данными независимо: после повторного наполнения

user-service идентификаторы начинаются заново, а в проекции остаются строки прежнего прогона с теми же идентификаторами. Событие приводило не к вставке, а к обновлению существующей строки, и счётчик не менялся.

Проверка переписана на содержательную: сценарий создаёт комментарий от нового пользователя и убеждается, что в ответе указано его имя. Имя автора social-service берёт исключительно из user_projection и при её отсутствии подставляет заглушку вида user<id>, поэтому совпадение имени доказывает, что событие user.registered дошло и было применено. Существование проекции рецепта доказывается уменьшением счётчика при удалении: уменьшиться может только запись, которая была создана событием recipe.published. Наблюдение по абсолютным значениям заменено на относительное и в проверке счётчика лайков — рецепт мог собрать реакции раньше.

```
$ node events-smoke.mjs
EVENTS: RabbitMQ-взаимодействие @ http://localhost:3000

1. Брокер подключён во всех сервисах      ☐ user/recipe/social → connected
2. user.registered → проекция пользователей ☐ имя автора взято из проекции
3. recipe.published → проекция рецептов     ☐ рецепт опубликован
4. social.recipe.liked → счётчик лайков      ☐ recipe-service увеличил likesCount
                                           ☐ recipe-service уменьшил likesCount
5. social.comment.* → счётчик комментариев ☐ увеличил и уменьшил commentsCount
6. recipe.deleted → каскадная очистка       ☐ проекция существовала и удалена
                                           ☐ избранное очищено без внешних ключей
7. Идемпотентность                          ☐ повторный лайк не публикует второе событие
                                           ☐ счётчик вырос ровно на 1

ИТОГО: 22 успешно, 0 провалено
```

Листинг 7 — результат проверки событийного взаимодействия

Журналы обработанных событий подтверждают работу защиты от повторов: в базе social-service накопилось 13 записей processed_events, в базе recipe-service — 6, а dead-letter-очереди остались пустыми.

10. Выводы

Подключён и настроен RabbitMQ, реализовано межсервисное взаимодействие через topic-exchange с двумя очередями потребителей и dead-letter-контуром. Событиями решены две задачи, которые в монолите решались внешними ключами и транзакциями: поддержание денормализованных счётчиков и каскадное удаление связанных данных между разными базами.

Обеспечены идемпотентность обработки, защита от устаревших событий, автоматическое переподключение и деградация при недоступности брокера. Корректность подтверждена 22 проверками сквозного сценария на реальном брокере в контуре docker compose.